

User Manual

This manual describes how to use the Bug Report Classification tool, which classifies GitHub bug reports as either performance-related (class 1) or non-performance-related (class 0).

1. Project Structure

```
bug_classification/
  ■■■ classify.py # Main experiment script
  ■■■ data/ # Place CSV datasets here (already placed for you)
    ■■■■ pytorch.csv
    ■■■■ tensorflow.csv
    ■■■■ keras.csv
    ■■■■ incubator-mxnet.csv
    ■■■■ caffe.csv
  ■■■ results/ # Auto-generated: raw_results.csv, summary_table.csv ■■■ figures/
  # Auto-generated: PNG figures
```

2. Installation

Step 1 — Ensure Python 3.9+ is installed:

```
python --version
```

Step 2 — Install dependencies:

```
pip install pandas numpy scikit-learn xgboost scipy nltk matplotlib seaborn
```

Step 3 — Place the dataset CSV files inside the **data/** folder.

3. Running the Experiments

Navigate to the project root and run:

```
python classify.py
```

The script will:

1. Load and preprocess each dataset.
2. Run 30 repeated stratified train/test splits (70/30) for three models: NB (baseline), XGB-TF-IDF (ablation), and XGBoost+Features (proposed).
3. Compute Accuracy, Precision, Recall, F1, and AUC for each run.
4. Perform Wilcoxon signed-rank tests: Proposed vs NB, and Proposed vs Ablation.
5. Save raw results to **results/raw_results.csv**.
6. Save a summary table to **results/summary_table.csv**.
7. Generate three figures to the **figures/** folder.

4. Output Files

results/raw_results.csv — All 30 repeat values per metric per model per project (columns: NB, XGBoost_ablation, XGBoost_proposed).

results/summary_table.csv — Mean $\pm$ std per metric per model per project.

figures/fig1_metric_comparison.png — Bar chart of all metrics.

figures/fig2_f1_boxplot.png — Box plots of F1 score across 30 repeats.

figures/fig3_training_time.png — Training time comparison.

5. Configuring the Experiment

You can modify the following variables at the top of **classify.py**:

- **PROJECTS** — list of project names to include (must match CSV filenames in data/).
- **REPEAT** — number of repeated train/test splits (default: 30).
- XGBoost hyperparameters (`n_estimators`, `max_depth`, `learning_rate`) in the `XGBClassifier` block.

7. Interpreting Results

The Wilcoxon test p-values are printed to the terminal and annotated on Figure 2. A p-value < 0.05 indicates a statistically significant difference. Two comparisons are made: XGBoost (Proposed) vs NB Baseline, and XGBoost (Proposed) vs XGB-TF-IDF Ablation.